Evolution of the Graphics Processing Unit (GPU)

Summary

W. J. Dally, S. W. Keckler, and D. B. Kirk. *IEEE Micro* 41(6), 42–51, 2021.

Context

Le GPU sono passate da unità finalizzate al rendering 3D per il gaming a GPU che integrano componenti specifici per l'AI, come i Tensor Core. L'hardware e il software (tramite CUDA, ad esempio) hanno seguito le esigenze e il loro cambiamento nel tempo.

The central argument is that programmability enables new applications, which then motivate specialized hardware and software. The examples and performance figures refer to the period covered by the 2021 article.

Origin of the first GPU: GeForce 256

Early 3D accelerators performed fragment operations such as rasterization, texture mapping, Z-buffering, and shading, while vertex transformation, clipping, and lighting remained on the CPU. In 1999, **GeForce 256** integrated vertex transformation and lighting with fragment processing, supporting more complex geometry while offloading the CPU. Its pipeline was still fixed-function. Demand for richer effects subsequently led to programmable vertex shaders in GeForce 3 and fragment shaders in GeForce FX.

Programmability and HPC

Early general-purpose GPU computing (GPGPU) exploited programmable shaders' floating-point throughput, but required scientific algorithms to be expressed as rendering tasks. Fragment shaders read textures and interpolated attributes and wrote to the frame buffer. Stream-processing systems such as Brook-GPU helped abstract these restrictions and influenced later designs.

G80 (Tesla) and CUDA, launched in 2006, made general-purpose computing directly accessible. Streaming multiprocessors ran both graphics shaders and computational threads; shared memory supported communication within cooperative thread arrays. CUDA provided a C-like, thread- and data-parallel programming system, removing the need to encode numerical work as graphics.

Fermi strengthened FP64 support and reliability through ECC on main memory and parity protection for on-chip memories. These features addressed accuracy and long-running HPC workloads. CUDA Fortran, OpenACC, and libraries such as cuBLAS, cuSPARSE, and cuFFT reduced porting effort. Together with education and reusable software, they enabled large scientific systems such as Titan and Summit.

Deep learning revolution and Tensor Cores

Deep learning required algorithms, datasets, and enough computing power to train networks in practical time. GPUs supplied the computational resource: AlexNet was trained on two GeForce GTX 580 GPUs in 2012. cuDNN then supplied optimized neural-network primitives. Unlike many FP64-oriented HPC workloads, many deep learning operations, especially inference, can use lower precision to obtain greater throughput, provided accuracy and dynamic range remain adequate.

The article reports a 317× increase in GPU inference performance from Kepler in 2012 to Ampere in 2020. This trend, called **Huang's law**, reflects specialized data types and operations beyond transistor scaling alone. It is not a replacement for Moore's law or equivalent growth in every FP32 application.

Pascal added FP16 arithmetic and dot-product instructions. Volta introduced **Tensor Cores** for mixed-precision matrix multiply-and-accumulate operations, such as $D = AB + C$, using FP16 multiplication and FP32 accumulation. Performing many arithmetic operations per specialized instruction amortizes instruction-fetch, decode, and operand-handling overhead. Turing added integer Tensor Core operations, including INT8, particularly useful for inference. This combines a programmable architecture with efficient domain-specific hardware.

Horizontal scalability

Larger models require both *scale-up*, improving one GPU, and *scale-out*, distributing work across GPUs. **NVLink** and **NVSwitch** connect GPUs within nodes, while **InfiniBand** connects nodes into clusters. Selene illustrates this hierarchy with 560 eight-GPU nodes, totaling 4,480 GPUs. Efficient communication is essential to benefit from the additional arithmetic resources.

Software also determines realized performance: TensorRT optimizes inference, Triton manages serving, and CUTLASS supports matrix kernels. The article reports training benchmark improvements of up to 2.1× on unchanged hardware from software enhancements alone.

Ampere A100 as an architectural case study

The A100 extends Tensor Cores to formats including BFloat16, TF32, and FP64. TF32 retains FP32's exponent range with reduced significand precision. **Structured sparsity**, requiring two zeros per group of four weights, can double effective throughput by skipping supported zero-valued work. This requires the appropriate pattern and model preparation; it is not an automatic benefit for arbitrary sparse matrices.

The distinction between dense and sparse rates matters: A100's FP16 rates are 312 TFLOPS dense and 624 TFLOPS structured-sparse, compared with V100's 125 TFLOPS dense. A100 also improves bandwidth and cache capacity, direct transfers into shared memory, cache residency control, compression, and task synchronization. Separately, the Ampere graphics family integrates RT Cores for ray tracing and uses Tensor Cores for DLSS; these graphics features are not features of A100 itself.

Memory bottlenecks and the road ahead

Many HPC applications need approximately 1–4 bytes from main memory per FP64 operation, while A100 provides about 0.1 byte per peak FP64 operation. Without sufficient reuse, memory bandwidth therefore limits performance. The authors identify improved memory technology and packaging, larger clusters, domain-specific acceleration, and better programming abstractions as future directions. Their 2021 outlook also emphasizes nonuniform access costs and placing computation near its data.

Main contribution and perspective

The contribution is a historical explanation of how programmability, specialization, data movement, and software jointly drove GPU adoption. Its scope is predominantly NVIDIA's evolution, so its examples are not a comparative survey of all GPU vendors.